

Source-to-Image (S2I)

Modulo: 3 — Deploy di Applicazioni

Versione: OpenShift 4.14+ su Azure ARO

Obiettivi di Apprendimento

- Comprendere l'architettura S2I e il flusso completo: sorgente → build → immagine → deploy
 - Usare `oc new-app` con un repository Git per costruire e deployare un'applicazione
 - Creare e gestire una `BuildConfig` con le diverse strategie di build (Source, Docker)
 - Configurare trigger di build automatici (ImageChange, ConfigChange, Webhook)
 - Monitorare, avviare e annullare build con i comandi `oc start-build`, `oc logs`, `oc cancel-build`
 - Personalizzare il processo S2I tramite variabili di ambiente e file `.s2i/environment`
-

1. Teoria e Concetti Chiave

Cos'è Source-to-Image (S2I)

Source-to-Image (S2I) è un framework di build sviluppato da Red Hat che permette di costruire immagini container direttamente dal codice sorgente, **senza scrivere un Dockerfile**.

Il processo S2I è composto da tre fasi:

1. **Download del sorgente:** OCP scarica il codice da un repository Git
2. **Build:** il codice viene compilato/assemblato all'interno di una **builder image** specializzata per il linguaggio/framework
3. **Produzione dell'immagine:** viene creata una nuova immagine container che include il sorgente compilato + il runtime

Vantaggi di S2I rispetto al Dockerfile tradizionale:

Aspetto	Dockerfile	S2I
Scrittura richiesta	Dockerfile manuale	Nessun file necessario
Sicurezza	Dipende dallo sviluppatore	Builder image curata da Red Hat
Riproducibilità	Variabile	Standardizzata
Aggiornamenti runtime	Manuale	Trigger automatico via ImageChange
Curva di apprendimento	Richiede conoscenza Docker	Minimale

Builder Images Disponibili in OCP

OCP include builder image ufficiali Red Hat per i principali linguaggi:

Linguaggio/Framework	ImageStream Name	Versioni tipiche
Node.js	<code>nodejs</code>	18, 20
Python	<code>python</code>	3.9, 3.11
Java (OpenJDK)	<code>java</code>	11, 17, 21
PHP	<code>php</code>	8.0, 8.1
Ruby	<code>ruby</code>	3.0, 3.1
.NET	<code>dotnet</code>	6.0, 8.0
Go	<code>golang</code>	1.19, 1.21
Perl	<code>perl</code>	5.32

```
# Visualizzare tutte le builder image disponibili nel namespace 'openshift'  
oc get imagestreams -n openshift
```

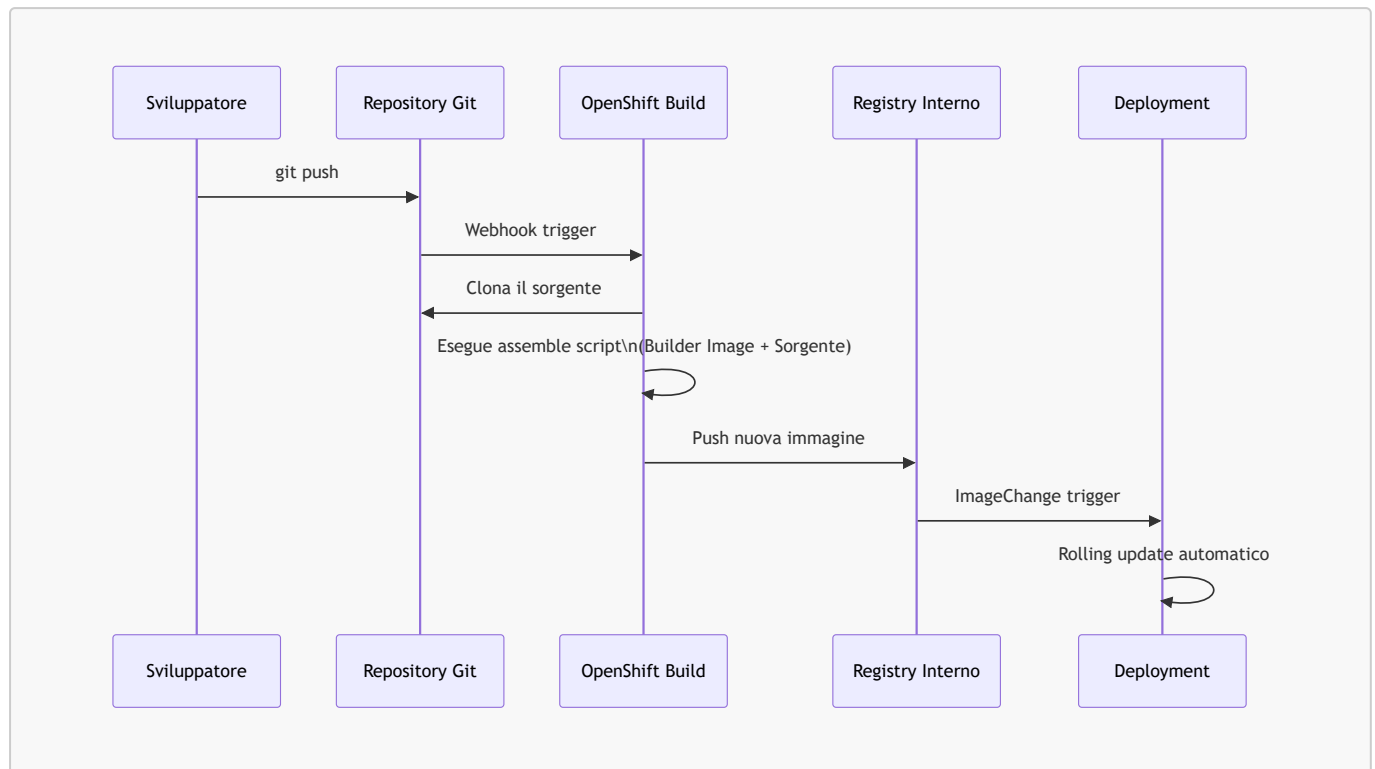
```
# Visualizzare i tag disponibili per una builder image specifica  
oc describe imagestream nodejs -n openshift
```

Struttura di una BuildConfig

La **BuildConfig** è la risorsa OCP che definisce come costruire un'immagine. Contiene:

- **source:** dove si trova il codice (Git URL, branch, sottodirectory)
 - **strategy:** come costruire (Source/S2I, Docker, Custom)
 - **output:** dove salvare l'immagine prodotta (ImageStream interno o registry esterno)
 - **triggers:** quando avviare automaticamente una nuova build
-

2. Diagramma — Flusso S2I End-to-End



3. Comandi Pratici con Output Atteso

```

# Creare un'applicazione da repository Git – OCP rileva il linguaggio
automaticamente
oc new-app https://github.com/openshift/nodejs-ex.git --name=nodejs-app -n my-
project

# Output atteso:
# --> Found image 9b3b4e7 (3 weeks old) in image stream "openshift/nodejs" under
tag "20-ubi9"
# --> Creating resources ...
#   imagestream.image.openshift.io "nodejs-app" created
#   buildconfig.build.openshift.io "nodejs-app" created
#   deployment.apps "nodejs-app" created
#   service "nodejs-app" created
# --> Success
#   Build scheduled, use 'oc logs -f buildconfig/nodejs-app' to track its
progress.

# Seguire i log della build in tempo reale
oc logs -f buildconfig/nodejs-app -n my-project

# Output atteso (estratto):
# Cloning "https://github.com/openshift/nodejs-ex.git" ...
# Generating dockerfile with builder image image-registry.../nodejs:20-ubi9
# STEP 1/9: FROM image-registry.openshift-image-
registry.svc:5000/openshift/nodejs:20-ubi9

```

```
# STEP 2/9: LABEL ...
# ...
# Successfully pushed image-registry.openshift-image-registry.svc:5000/my-
project/nodejs-app:latest
# Push successful

# Verificare lo stato della BuildConfig
oc get buildconfig nodejs-app -n my-project

# Output atteso:
# NAME          TYPE      FROM    LATEST
# nodejs-app    Source    Git     1

# Elencare le build eseguite
oc get builds -n my-project

# Output atteso:
# NAME          TYPE      FROM          STATUS      STARTED          DURATION
# nodejs-app-1   Source    Git@abc1234    Complete    2 minutes ago    45s

# Avviare manualmente una nuova build
oc start-build nodejs-app -n my-project

# Output atteso:
# build.build.openshift.io/nodejs-app-2 started

# Avviare una build da un commit specifico
oc start-build nodejs-app --commit=abc1234 -n my-project

# Annullare una build in corso
oc cancel-build nodejs-app-2 -n my-project

# Output atteso:
# build.build.openshift.io/nodejs-app-2 cancelled

# Descrivere la BuildConfig per vedere tutti i dettagli e i trigger
oc describe buildconfig nodejs-app -n my-project

# Specificare esplicitamente il linguaggio/tag S2I
oc new-app nodejs:20~https://github.com/openshift/nodejs-ex.git \
  --name=nodejs-app-v20 \
  -n my-project

# Impostare variabili di ambiente nella BuildConfig
oc set env buildconfig/nodejs-app \
  NPM_MIRROR=https://registry.npmjs.org \
  NODE_ENV=production \
  -n my-project
```

4. Esempi YAML Completi

BuildConfig con strategia Source (S2I)

```
# file: buildconfig-nodejs.yaml
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: nodejs-app
  namespace: my-project
  labels:
    app: nodejs-app
spec:
  # Sorgente del codice
  source:
    type: Git
    git:
      uri: https://github.com/openshift/nodejs-ex.git
      ref: main # branch, tag o commit SHA
      contextDir: / # sottodirectory del repository (/ = root)
  # Strategia di build S2I
  strategy:
    type: Source
    sourceStrategy:
      from:
        kind: ImageStreamTag
        namespace: openshift # namespace dove risiede la builder image
        name: "nodejs:20-ubi9" # builder image e tag
      env:
        # variabili passate durante la build
        - name: NPM_MIRROR
          value: https://registry.npmjs.org
        - name: NODE_ENV
          value: production
  # Immagine di output
  output:
    to:
      kind: ImageStreamTag
      name: "nodejs-app:latest" # ImageStream di destinazione (nello stesso namespace)
  # Trigger automatici
  triggers:
    - type: ImageChange # nuova build se la builder image viene aggiornata
      imageChange: {}
    - type: ConfigChange # nuova build se questa BuildConfig viene modificata
    - type: GitHub # webhook GitHub
      github:
        secret: my-github-secret
    - type: Generic # webhook generico (GitLab, Bitbucket, ecc.)
      generic:
        secret: my-generic-secret
```

```
# Impostazioni di run
runPolicy: Serial          # le build vengono eseguite in sequenza (non in
                             parallelo)
successfulBuildsHistoryLimit: 5  # conserva le ultime 5 build riuscite
failedBuildsHistoryLimit: 3     # conserva le ultime 3 build fallite
```

BuildConfig con strategia Docker

```
# file: buildconfig-docker.yaml
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: myapp-docker
  namespace: my-project
spec:
  source:
    type: Git
    git:
      uri: https://github.com/myorg/myapp.git
      ref: main
  strategy:
    type: Docker          # usa il Dockerfile nel repository
    dockerStrategy:
      dockerfilePath: Dockerfile  # percorso del Dockerfile (relativo a
contextDir)
      env:
        - name: BUILD_ENV
          value: production
  output:
    to:
      kind: ImageStreamTag
      name: "myapp-docker:latest"
  triggers:
    - type: ConfigChange
    - type: GitHub
    github:
      secret: my-github-secret
```

Configurare un Webhook GitHub

```
# Recuperare il segreto e l'URL del webhook GitHub
oc describe buildconfig nodejs-app -n my-project | grep -A5 "Webhook GitHub"

# Output atteso:
# Webhook GitHub:
#   URL:
https://api.cluster.example.com:6443/apis/build.openshift.io/v1/namespaces/my-
project/buildconfigs/nodejs-app/webhooks/my-github-secret/github
#   Secret: my-github-secret

# Creare il Secret per il webhook (se non già presente)
oc create secret generic my-github-secret \
  --from-literal=WebHookSecretKey=my-super-secret-token \
  -n my-project
```

File `.s2i/environment` nel Repository

```
# Creare nel repository Git il file .s2i/environment per variabili di build
# Questo file viene letto dalla builder image durante l'assemble

# Contenuto di .s2i/environment nel repository:
NODE_ENV=production
NPM_MIRROR=https://registry.npmjs.org
HTTP_PROXY=http://proxy.company.com:3128
HTTPS_PROXY=http://proxy.company.com:3128
```

5. Note Specifiche per Azure ARO

```
# Su ARO con Azure Repos (Azure DevOps Git), configurare S2I con autenticazione
SSH

# 1. Creare un Secret SSH con la chiave privata
oc create secret generic azure-repos-ssh \
  --from-file=ssh-privatekey=~/.ssh/id_rsa \
  --type=kubernetes.io/ssh-auth \
  -n my-project

# 2. Aggiungere il Secret come source secret nella BuildConfig
oc set build-secret --source buildconfig/nodejs-app azure-repos-ssh -n my-project

# 3. Usare l'URL SSH del repository Azure Repos
oc new-app "nodejs:20~git@ssh.dev.azure.com:v3/myorg/myproject/myrepo" \
  --source-secret=azure-repos-ssh \
  --name=nodejs-app \
  -n my-project
```

⚠️ Attenzione ARO: Su ARO, le build S2I vengono eseguite con il SCC `restricted-v2`. La builder image non deve richiedere privilegi root. Le builder image ufficiali Red Hat (nodejs, python, java, ecc.) sono già compatibili con questo vincolo.

⚠️ Attenzione ARO: Se il cluster ARO è dietro un proxy HTTP aziendale, configurare le variabili `HTTP_PROXY`, `HTTPS_PROXY` e `NO_PROXY` nel BuildConfig o in `.s2i/environment`. Il proxy è spesso necessario per scaricare dipendenze npm, pip, maven, ecc.

Aspetto	OCP On-Premise	Azure ARO
Registry di output	Registry interno OCP	Registry interno OCP (su Azure Blob)
Registry privato sorgente	Qualsiasi	Consigliato ACR + autenticazione
Git privato	SSH key / HTTP basic	Azure Repos: SSH key o PAT
Proxy	Opzionale	Spesso necessario in ambienti enterprise
Builder image	Gestite dall'admin	Pre-caricate nel namespace <code>openshift</code>

6. Riepilogo dei Punti Chiave

- **S2I** permette di costruire immagini container direttamente dal codice sorgente Git, senza Dockerfile
 - `oc new-app <linguaggio>~<git-url>` avvia il processo S2I usando la builder image specificata
 - La **BuildConfig** definisce sorgente, strategia, output e trigger della build
 - I **trigger** (`ImageChange`, `ConfigChange`, `Webhook`) permettono build automatiche senza intervento manuale
 - `oc logs -f buildconfig/<nome>` mostra i log della build in tempo reale per diagnosticare problemi
 - Il file `.s2i/environment` nel repository Git è il modo più pulito per passare variabili di build
 - Su **ARO con Azure Repos**, usare un Secret SSH o PAT per autenticare l'accesso al repository privato
-

7. Domande di Verifica

1. Qual è la differenza principale tra una build S2I (Source strategy) e una build Docker strategy?

- A) La strategia Docker è più veloce
- B) **La strategia Source non richiede un Dockerfile: usa una builder image specializzata per assemblare il codice** ✓
- C) La strategia Source funziona solo con Node.js
- D) Non c'è differenza pratica

2. Quale risorsa OCP viene creata da `oc new-app` che contiene la definizione del processo di build?

- A) BuildRun
- B) Pipeline
- C) **BuildConfig** ✓
- D) ImageBuildRequest

3. Quale trigger nella BuildConfig causa l'avvio automatico di una nuova build quando la builder image viene aggiornata?

- A) ConfigChange
- B) **ImageChange** ✓
- C) GitHub
- D) Generic

4. Come si seguono i log di una build in esecuzione?

- A) `oc get build nodejs-app-1 -o logs`
- B) `oc describe buildconfig nodejs-app`
- C) **`oc logs -f buildconfig/nodejs-app`** ✓
- D) `oc build-logs nodejs-app`

5. Su ARO, come si configura S2I per costruire da un repository Azure Repos privato?

- A) Inserendo username e password direttamente nell'URL Git
- B) Creando un ConfigMap con le credenziali
- C) **Creando un Secret di tipo `kubernetes.io/ssh-auth` e configurandolo come source secret nella BuildConfig** ✓
- D) Non è possibile: S2I supporta solo GitHub e GitLab